Trabajo Práctico 4 - La Capa de Presentación

Ejercicio 1: Manipulación del DOM con JavaScript

Objetivo: Practicar la selección y modificación de elementos HTML utilizando JavaScript para entender cómo se puede alterar dinámicamente el contenido de una página.

1. Crea un archivo `index.html` que contenga:
   - Un título `<h1>` con un `id="titulo"`.
   - Un párrafo `<p>` con una clase `descripcion`.
   - Un botón `<button>` con un `id="cambiarTexto"`.
   - Un contenedor `<div>` vacío con un `id="contenido-dinamico"`.
2. Dentro del mismo archivo, agrega una etiqueta `<script>` al final del `<body>`.
3. En el script:
   - Usa `document.getElementById` para seleccionar el título y cambia su texto a "¡Página Interactiva!".
   - Usa `document.querySelector` para seleccionar el párrafo por su clase y añade un borde de 1px sólido con CSS mediante la propiedad `style`.
   - Añade un "event listener" al botón para que, al hacer clic, el `innerHTML` del `div` con `id="contenido-dinamico"` se rellene con una lista no ordenada `<ul>` con tres elementos `<li>`.

Ejercicio 2: Creación y Eliminación de Elementos

Objetivo: Aprender a crear, añadir y eliminar elementos del DOM de forma dinámica en respuesta a las acciones del usuario.

1. Crea un archivo HTML con:
   - Una lista no ordenada `<ul id="lista-items">`.
   - Un campo de texto `<input type="text" id="nuevo-item-texto">`.
   - Un botón `<button id="agregar">` para añadir elementos.
   - Un botón `<button id="eliminar">` para quitar el último elemento.
2. Escribe un script que:
   - Al hacer clic en el botón "agregar", lea el valor del campo de texto.
   - Cree un nuevo elemento `<li>` con el texto introducido.
   - Añada el nuevo `<li>` al final de la lista `<ul>`.
   - Limpie el campo de texto después de añadir el elemento.
   - Al hacer clic en el botón "eliminar", quite el **último** `<li>` de la lista. Si la lista está vacía, no debe hacer nada.

Ejercicio 3: Estructura y Estilo con CSS y Box Model

Objetivo: Aplicar estilos con CSS, comprendiendo el modelo de caja (Box Model) para controlar el espaciado y el tamaño de los elementos.

1. Crea una estructura de archivos:

```
/ejercicio3
    index.html
    styles.css
```

2. En `index.html`, crea tres `<div>` con la clase `caja`. Dentro de cada uno, pon un título `<h2>` y un párrafo `<p>`.
3. En `styles.css`, enlaza el archivo a tu `index.html` y define los estilos para la clase `.caja`:
   - Un `width` de `300px`.
   - Un `background-color` diferente para cada caja (puedes usar `:nth-child(1)`, `:nth-child(2)`, etc.).
   - Un `padding` de `15px`.
   - Un `border` de `2px solid black`.
   - Un `margin` de `20px`.
   - Observa en las herramientas de desarrollador de tu navegador cómo se visualiza el modelo de caja para cada `div`.

Ejercicio 4: Layout con Flexbox

Objetivo: Utilizar el modelo de layout Flexbox para crear alineaciones y distribuciones de elementos de forma flexible y responsiva.

1. Crea un `index.html` con un `div` contenedor (`class="flex-container"`) y cinco `divs` hijos dentro (`class="flex-item"`).
2. En un archivo `styles.css` asociado:
   - Establece el `display` del `.flex-container` a `flex`.
   - Experimenta con la propiedad `flex-direction`: primero `row` y luego `column`.
   - Usa `justify-content` para alinear los ítems. Prueba los valores: `flex-start`, `center`, `space-between`, y `space-around`.
   - Usa `align-items` para alinear los ítems en el eje transversal. Prueba `stretch`, `center`, y `flex-start`.
   - Añade la propiedad `gap` para crear un espaciado uniforme entre los ítems.

Ejercicio 5: Diseño de Layout con CSS Grid

Objetivo: Implementar un layout de página web complejo y estructurado utilizando CSS Grid.

1. Crea un `index.html` con la siguiente estructura semántica:

```html
<div class="grid-container">
  <header class="grid-item header">Header</header>
  <nav class="grid-item sidebar">Sidebar</nav>
  <main class="grid-item content">Main Content</main>
  <footer class="grid-item footer">Footer</footer>
</div>
```

2. En `styles.css`, diseña el layout usando CSS Grid:
   - Aplica `display: grid` al `.grid-container`.
   - Define una estructura de 2 columnas y 3 filas usando `grid-template-columns` (ej. `1fr 3fr`) y `grid-template-rows` (ej. `auto 1fr auto`).
   - Asigna cada elemento a su área correspondiente usando la propiedad `grid-column` y `grid-row`.
   - (Opcional Avanzado) Define el mismo layout utilizando `grid-template-areas` para un código más semántico.

Recursos Adicionales

- MDN HTML[1]
- MDN CSS[2]
- MDN JavaScript[3]
- A Complete Guide to Flexbox (CSS-Tricks)[4]
- A Complete Guide to Grid (CSS-Tricks)[5]

Trabajo de Cursada: Construyendo la Interfaz de Usuario

Objetivo: Crear una interfaz de usuario (frontend) en HTML, CSS y JavaScript que consuma la API REST que construiste en el práctico anterior.

En este paso, te enfocarás exclusivamente en el cliente. Tu servidor Go con la API REST debe estar corriendo en segundo plano.

1. **Estructura HTML**:
   - Crea un nuevo `index.html` (puedes servirlo con un servidor de archivos estáticos simple o abrirlo directamente en el navegador).
   - Diseña la estructura de la página:
     - Un título principal.
     - Un formulario para crear nuevas entidades (con campos de `input` y un botón de envío).

[1] https://developer.mozilla.org/es/docs/Web/HTML
[2] https://developer.mozilla.org/es/docs/Web/CSS
[3] https://developer.mozilla.org/es/docs/Web/JavaScript
[4] https://css-tricks.com/snippets/css/a-guide-to-flexbox/
[5] https://css-tricks.com/snippets/css/complete-guide-grid/

- Un contenedor (un `<div>` o `<ul>`) donde se mostrará la lista de entidades. Asígnale un `id` para poder seleccionarlo con JavaScript.

2. **Comunicación con la API (JavaScript)**:
   - Crea un archivo `app.js` y enlázalo a tu `index.html`.
   - Escribe una función que, al cargar la página, haga una petición `fetch` a tu endpoint `GET /<entidades>`.
   - Cuando reciba los datos, debe generar dinámicamente los elementos HTML (por ejemplo, `<li>` o `<div>`) para cada entidad y agregarlos al contenedor del DOM. Cada elemento debe mostrar la información relevante y tener un botón para eliminarlo.
   - Añade un "event listener" al formulario para que, al enviarse, prevenga el comportamiento por defecto, tome los datos de los inputs, y haga una petición `POST` con `fetch` al endpoint de creación.
   - Después de crear un nuevo elemento, la aplicación debe volver a pedir la lista completa y refrescar la vista para mostrar el nuevo ítem.
3. **Implementación del Borrado**:
   - Añade un "event listener" a los botones de borrado. Como los botones se crean dinámicamente, deberás usar delegación de eventos.
   - Al hacer clic, el listener debe obtener el ID del elemento y realizar una petición `DELETE` con `fetch` al endpoint correspondiente.
   - Tras un borrado exitoso, refresca la lista para que el elemento desaparecido ya no se muestre.
4. **Estilos con CSS**:
   - Crea un archivo `styles.css` y enlázalo.
   - Utiliza Flexbox o CSS Grid para organizar los elementos principales de tu página (formulario y lista).
   - Añade estilos básicos para que la aplicación sea visualmente agradable y fácil de usar.